Software Engineering - Assignment #03

Smart Campus Room Availability Checker

Report: scope, criteria, traceability, tests, issues, agent analysis, reflection

1. Task 1 - Baseline and plan

Scope. One responsive page for one room (Room B-204) with fictional data. The page only checks whether a requested time slot is free; it never creates or confirms a booking. Login, database, backend, external APIs and payments are excluded.

Acceptance criteria (own words).

• AC1: times are whole minutes after midnight, 0 <= start < end <= 1440, and start is strictly later than now.

• AC2: a slot may last at most 120 minutes; exactly 120 is fine.

• AC3: a blocked room rejects every request.

• AC4: a slot that overlaps any existing booking is rejected; slots that only touch at an endpoint are allowed.

• AC5: the page shows a clear Available or Unavailable result and never changes existing bookings.

• AC6: invalid input gives an understandable message and the layout works on a narrow phone screen.

Function contract. `can_book(start, end, now, blocked, existing) -> bool`. Inputs: integers start, end, now; boolean blocked; list of (start, end) tuples. Returns True only if the request is valid under AC1-AC4. It does not modify its inputs.

Likely failure points. (1) Endpoint overlap: 660-720 against 600-660 must be Available, so the overlap test must use strict comparisons. (2) Starts-now: start == now must be rejected. (3) Exactly 120 vs 121 minutes. (4) Non-integer or boolean input slipping through because Python compares floats and True/False like numbers.

Assumptions and manual checks. now is valid (0-1439); all data is fictional; a booking list item is a 2-tuple. Manually verified: layout at 390 px width, wording of messages, that no network request is made by the page.

2. Task 2 - Python implementation (booking.py, test_booking.py)

The assistant first proposed a short plan (check blocked, validate bounds, check now, check duration, loop over bookings with *start* < *b_end and b_start* < *end*). The first generated implementation was saved unchanged as **booking_v1.py**. Branch review of v1:

• `if blocked` returns False - AC3.

• `start < 0 or end > 1440 or start >= end` - AC1 bounds and ordering.

• `start <= now` - AC1 strictly after now.

• `end - start > 120` - AC2 (120 allowed).

• `start < e and s < end` - AC4 (strict, so touching is allowed).

• AC5 is a UI rule; the function only returns True/False and never mutates the list.

3. Task 3 - Prototype

The prototype is the single file **index.html** (plain HTML, CSS and JavaScript, no backend, no database, no login). **Important disclosure:** this prototype was generated with a general AI assistant (Claude), not with Bolt.new. [If your course requires Bolt.new: generate it there with the prompt below, save the screenshot and preview link, and replace this paragraph.]

Prompt to use in Bolt.new: *Build a one-page, responsive Smart Campus Room Availability Checker for one room with fictional data. Inputs: start and end time (HH:MM). Rules: 0 <= start < end <= 1440 minutes, start strictly after now (09:00), duration at most 120 minutes (exactly 120 allowed), blocked room rejects everything, overlap with existing bookings rejected but touching endpoints allowed. Show a clear Available / Unavailable result and validation messages. Do not mutate bookings. No login, backend, database, external API or payments.*

Main parts of index.html: **canBook()** (same rules as booking.py), **parseTime()** (input parsing and validation), **run()** (reads the form, shows result or error), and the CSS block with a mobile-first layout.

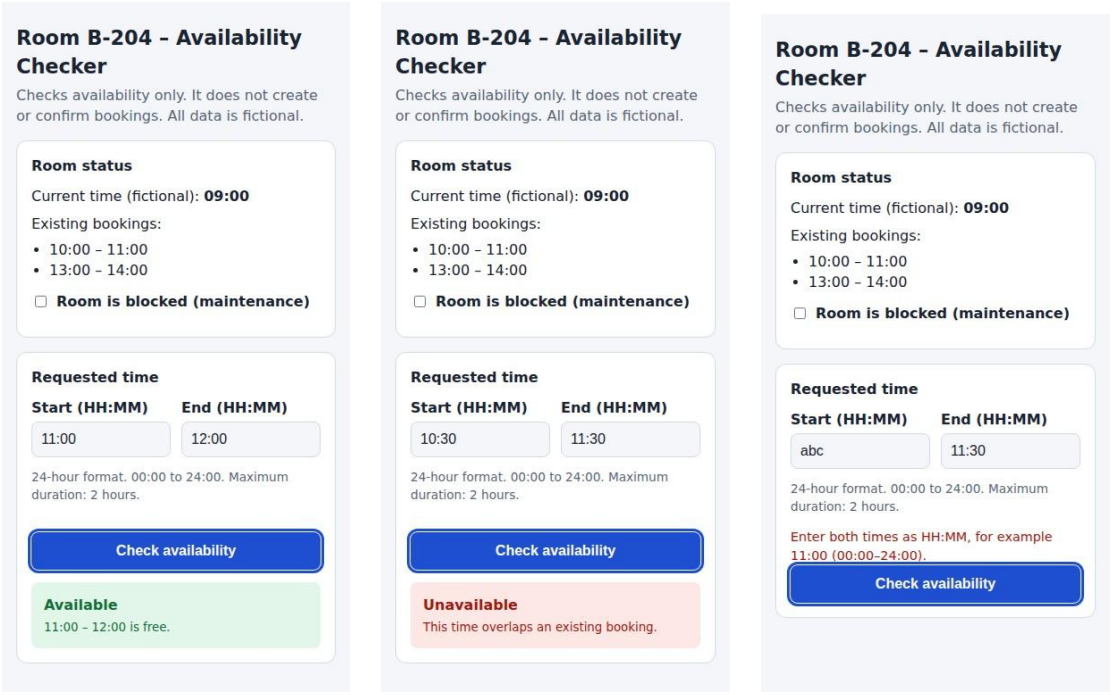


Figure 1: prototype at 390 px width - Available, Unavailable, validation error.

4. Task 4 - Test execution and traceability

Base data: now=540, blocked=False, existing=[(600,660)] unless stated. Results below were produced by actually running the code (unittest, 24 tests).

AC	Impl. / file	Test input	Expected	v1 actual	Final actual
AC1	booking.py	Starts now 540-570	Unavailable	Unavailable	Unavailable
AC1	booking.py	Reversed 720-660	Unavailable	Unavailable	Unavailable
AC1	booking.py	Day end 1380-1440	Available	Available	Available
AC1	booking.py	Past day 1400-1441	Unavailable	Unavailable	Unavailable
AC1	booking.py	Non-integer 660.5-700	Unavailable	Available (FAIL)	Unavailable
AC2	booking.py	Exactly 120 min 720-840	Available	Available	Available
AC2	booking.py	121 min 720-841	Unavailable	Unavailable	Unavailable
AC3	booking.py	Blocked 660-720	Unavailable	Unavailable	Unavailable
AC4	booking.py	Touching 660-720	Available	Available	Available
AC4	booking.py	Overlap 630-690	Unavailable	Unavailable	Unavailable
AC4	booking.py	Inside existing 610-650	Unavailable	Unavailable	Unavailable

AC5	index.html	UI shows Available / Unavailable	Clear result	Manual: pass	Manual: pass
AC6	index.html	390 px wide viewport; invalid text	Readable + message	Manual: pass	Manual: pass

v1 run: `Ran 24 tests in 0.001s FAILED (failures=1)`

Final run: `Ran 24 tests in 0.000s OK`

The test file also covers day bounds, zero length, multiple bookings, an empty booking list, now boundaries (0 and 1439), unchanged inputs and real bool return values. The JavaScript canBook() was run against the nine specification cases with Node.js: 9 of 9 passed.

5. Task 5 - Cleanup and retest

Issue	Reproduction	Priority	Prompt / change	Retest result
I1 (functional): v1 accepts non-integer times, violating AC1	can_book(660.5, 700, 540, False, [(600,660)]) returned True	High	Prompt: "Reject non-integer start, end and now (also bool) without changing any other rule." Added _is_minute() and a type check.	test_non_integer_inputs_rejected now passes; 24/24 pass
I2 (code quality): magic numbers 1440 and 120, no docstring	Code review of v1	Medium	Prompt: "Name the constants and add a docstring listing AC1-AC4; do not change behavior."	All 24 tests unchanged and passing
I3 (usability): page shows only a result with no reason	Manual use of the form	Medium	Added reasonFor() so an Unavailable result explains why; added parseTime() error text	Manual check at 390 px: pass

Before/after evidence for I1: v1 output `True` for 660.5 (failing test shown in the v1 run above); final output `False`. For I3, the Unavailable box in Figure 1 shows the new reason line. [Add your own before screenshot if your instructor requires image evidence for I3.]

6. Task 6 - Coding-agent analysis

Agent loop: user goal -> model/plan -> tool call -> observation -> next step -> human review. In this work: goal (implement can_book under AC1-AC6) -> plan (checks, assumptions, boundary tests) -> tool calls (create files, run unittest, run Node tests, render screenshots) -> observation (test output, one v1 failure) -> next step (targeted fix) -> human review of every diff and result before submission.

Context, tools, permissions (observed): the assistant read the uploaded assignment slides, wrote files in a sandbox, ran Python, Node.js and a headless browser, and had no network access for the code it ran. **Assumptions (not observed):** how the assistant selects context internally and how it ranks alternatives.

Safety checkpoint: before accepting generated changes I read the diff, ran the full test set and confirmed no network or file access code was added. **Link to dependability:** tests protect reliability, limited tool permissions reduce security risk, and the person who submits the work remains responsible for it.

7. Reflection (dependability and AI)

In this assignment I compared two ways of working. AI-assisted coding meant I wrote the contract and acceptance criteria first, asked the assistant for a small function, and checked every branch against AC1-AC4 with tests. Vibe coding would mean accepting generated code because it looks right. The difference showed up immediately: the first version passed all boundary tests, yet it still accepted a non-integer start such as 660.5, which only a test written from the criteria revealed. Reliability example: that float input violated AC1 until I added type validation. Usability example: the page shows a reason with every Unavailable result and rejects malformed text before checking. Security risk example: generated code could read unchecked input or add a backend I forbade, so inputs are validated and the page has no network

code. Code review, tool permissions and tests stay human responsibilities because the assistant cannot know what I intended, and only a person is accountable for what is submitted. I modified one suggestion: a shorter overlap check that used closed intervals, which would have wrongly rejected touching bookings. Next I would add more tests for the page itself without expanding scope.

(191 words)

8. AI use record

Tool	Model	Date	Used for
Claude (Anthropic)	As displayed in the chat interface	24 Oct 2026	Plan, booking_v1.py, booking.py, tests, index.html, report draft

AI_Evidence.pdf must contain at least four real prompt-response exchanges with your review decisions; paste your actual chat prompts and answers there.